Speaker Script

From 48% Loss to Zero · CEG5101 CA2 · Group 5 · Speaker A: Yang Kaiming (A0326309E) · Speaker B: Chen Danze (A0328226E) · 12 Nov 2026, 10:25–10:50, SDE1-SR1
12-minute talk. Orange = click cues. Grey boxes = notes, not to be read aloud. Slides 15–23 are only for Q&A. On slide 7 we switch to the live demo (`bash demo/live_demo.sh`); slide 8 is the recorded fallback.

From 48% Loss to Zero · Slide 1

[Speaker A · Kaiming] target 0:27 · slot 0:00–0:30

Good morning. I'm Kaiming, and this is Danze. Forty-eight percent: that is what a legitimate flow from h1 to h6 lost in its worst second, before our controller caught the flood on its link. About four seconds after the flood began, loss was zero, and it stayed zero. Today: those four seconds, and the design behind them.

> Gesture at the red 48% and at the red bars on the right. No separate transition line: the first sentence of slide 2 does that.

Task 1: One Mesh, One Flow to Protect, Three Attacks · Slide 2

[Speaker A · Kaiming] target 0:41 · slot 0:30–1:15

Here is the network, and the flow we protect. Host hi sits on switch si; each edge shows its cost C0, the link delay in milliseconds. Our flow, h1 to h6, takes s1, s2, s6 at cost 19, the blue line; the orange backup costs 22.

[click] Attack a, h2 to h3, uses s2–s3 and misses our path. [click] Attack b, h2 to h6, shares s2–s6: our running example. [click] Attack c, h5 to h6, takes s5, s3, s6: 16 beats 20.

Who decides where this flow goes? One loop in the controller.

> On the left: the same mesh built in MiniEdit in the Mininet 2.3.0 VM (loops silently, no click needed). Host links are 40 Mbps and 15 ms; hosts are 10.0.0.i/24.

Two Reference Controllers, Two Blind Spots, One 2-Second Loop · Slide 3

[Speaker A · Kaiming] target 0:38 · slot 1:15–2:00

We started from two reference controllers, each with one blind spot: one routes but cannot see an attack; the other blocks but never moves traffic.

[click] So we merged them into one loop, polled every 2 seconds: measure, detect, price, route. The design reason is one shared signal: links a blocked source used get A = 1, the red arrow. In case b, A = 1 lands on s2–s6.

First job of that loop: find the attacker, not the victim.

> Only if asked: the reference cost table is for an old 4-switch topology (D1); the reference detector updates before it compares, so its real threshold is about 21× the baseline (D5). Measure = FlowStats per source at its ingress switch and PortStats per link. Forwarding details (ARP proxy, flooding only to host ports, packet-in ignored until all 11 links are discovered, D11) are for Q&A.

> Short version if rehearsal runs long: "We merged the two reference controllers into one 2-second loop: measure, detect, price, route."

Task 2: Catch the Attacker, Not the Victim · Slide 4

[Speaker A · Kaiming] target 1:10 · slot 2:00–3:15

The design reason behind each detection rule is a concrete failure. Count all ICMP, and the victim's echo replies get h6 blocked, with our flow. So we skip echo replies.

[click] The reference detector updates before it compares: the real threshold becomes about 21 times the baseline, and the attack poisons that baseline. We compare first and never learn from an anomaly.

[click] A fixed 3x ignores how bursty a source is, so our threshold is the largest of 3b, b plus 4 sigma, and 500, after a warm-up. A block is a priority-5000 drop rule on all six switches.

On the right, case b in R1: h2 hit 4,994 packets per second, the top red point, against a threshold of 1,046, and was blocked. h6 was never flagged.

H1 → So the detector answers one question: who is attacking. How does routing use that answer? Danze?

> Not on screen: rules match ipv4_src (D2); each packet is counted once, at its ingress switch (D3); detector EMA weight 0.3, as in the reference code; auto-unblock after 10 s quiet (D8/D12).

Task 3: Price the Attack into the Link · Slide 5

[Speaker B · Danze] target 1:10 · slot 3:15–4:30

Thanks, Kaiming. Dijkstra only understands numbers, so we put the verdict into the link cost, back to s2–s6. Cost is C0, plus load weight alpha 10 times utilisation, plus anomaly weight beta 30 times the flag A. The load term reacts fast, but after the block it fades within a few polls. Beta times A is the memory: A stays 1 while the drop rule keeps counting, then 10 seconds more.

[click] On s2–s6: 1.19 before the flood, 3.67 at the block, then a plateau at 3.14.

[click] Beta must beat the 3-longer detour, our own load of about 2.7, and margin 2: about 7.7. Thirty leaves room.

[click] Testable prediction: each attacked link plateaus at C0 plus 30, over C0, the table on the right. We check it on slides 10 and 11.

A price alone moves nothing. When should a flow actually move?

> Why hold A (D7): the flood is dropped at the ingress, so R falls to 0 within a few polls. A stays 1 while the drop rate is at least 0.5T, then 10 s more. Why 0.5T (D8): the attacker's normal 1 pps ping also hits the drop rule. At the moment of the block the leftover flood load helps: R1 needed only $\beta > 3.2$, R3 > 6.9. From static costs alone: 22 − 19 + 2 = 5. Rmax = 40 Mbps; C0 rewritten for the 11 links (D1). s2-s6 returns to ×1.0 at 151.6 s, when h1 is still on the detour.

Task 4: Move Only When Clearly Cheaper, Install Egress First · Slide 6

[Speaker B · Danze] target 0:55 · slot 4:30–5:30

Every poll, each flow compares its path with the cheapest alternative. First, a what-if: we add the flow's own load to candidate links it does not use yet; otherwise h1's own 10 megabits make its current path look worse, and it flaps.

[click] Second, move only if at least 2 cheaper: that margin is our hysteresis.

[click] Third, egress first: s6, then s3, then the branch point s2 switches over; s1 is unchanged. No packet meets a missing rule.

[click] Case b in R1, the bars: stay 58.8, move 30.0, so it leaves; after the hold, 24.5 beats 30.1, so it returns. Link failures use the same rule.

H2 → That is the design. Does it work? Kaiming?

> Details: when leaving, s6 keeps its rule, s3 gets a new rule to s6, s2 is rewritten from →s6 to →s3, s1 is rewritten in place (next hop still s2). Nothing is deleted when leaving; the stale rule on s3 is deleted on the return.

Experiment: One Timeline, Three Cases, Three Runs · Slide 7

[Speaker A · Kaiming] target 0:29 · slot 5:30–6:00

Thanks, Danze. One run, about four minutes, three floods. After a 40-second baseline, cases a, b and c each get a 25-second flood and 45 seconds of recovery: the grey bands. h1 sends 10 megabits of UDP to h6 throughout, measured by iperf and tcpdump, not by the controller. Three runs.

Here is case b from a real run.

> **LIVE DEMO:** at this point switch to the terminal and run `bash demo/live_demo.sh` (or `quick`); narrate with docs/Live_Demo_Runbook_zh.pdf section 3, then come back to slide 9. Slide 8 (the recording) is the fallback if the live run fails (60-second rule). Instead of "Here is case b from a real run", say: "Let us run it now."

> Details: hping3 --icmp --flood -d 1000, about 4,700–4,800 pkt/s (≈39 Mbps) leaving the attacker; ping every 0.5 s; the other hosts send 1 pps background pings. The container has no link delays: routing is unaffected (it uses the C0 table), RTTs are just small. One timeline is 255 s. Short version: "Three 25-second floods on one timeline, measured with iperf and tcpdump, repeated three times."

Demo: Case b in a Real Run · Slide 8

[Speaker A · Kaiming] target 0:55 · slot 6:00–7:00

[click to play, 50 s]

[0:03] Normal: our flow on s1, s2, s6, no loss.

[0:09] The flood starts; loss appears here, bottom left.

[0:15 paused frame] One control step: h2 is over threshold and dropped on all switches, s2–s6 is marked, and our flow is on s1, s2, s3, s6.

[0:24] The drop counter keeps rising, [0:28] and tcpdump at h6 sees zero attack packets.

[0:33] Flood stops, ten-second hold, back to the short path.

[0:43 end card] Was that one lucky run? Here are all nine floods.

> **With the live demo this slide is skipped**; use it only as the fallback.

> Do not introduce the panes: the title card does it. If the speaker falls behind, the partner pauses with K or a mouse click on the video; the next click goes on to slide 9. If the video will not play: type 15 + Enter (stills), then the button there returns to slide 9. Values in the log lines are this recording's own; the numbers on the end card are the report's.

Detection and Blocking Results: One or Two Polls, Then Nothing Gets Through · Slide 9

[Speaker A · Kaiming] target 0:42 · slot 7:00–7:45

This result shows all nine floods were caught within one or two polls: 1.5 to 3.8 seconds, the table on the right. After each block, at most 101 in-flight packets arrived, within 28 milliseconds; no attack packet got through later than one second after the block. On the left, black is what reached the victim.

Why one or two polls? Samples come every 2.4 seconds; a flood that starts late in an interval gives a partial first sample.

H3 → So the attacker is stopped every time. Did our legitimate flow survive? Danze?

> If asked: 3,608–14,251 attack packets reached the victim before the block, roughly flood rate × (delay − 0.5 s), since hping3 starts about 0.5 s after the timing mark; less in b and c because part of the flood is dropped on the congested link. The drop rule kept counting (R1 case b: n_packets = 31,253 on s2). R1 b took 3.8 s because its first sample, 491 pkt/s, was under the 500 floor and was learned (backup slide 19). In c the marked links are s3-s5 and s3-s6, the links the attack used. VM with link delays: 2.2 / 1.6 / 3.8 s.

Case b: Block and Reroute in One Control Interval · Slide 10

[Speaker B · Danze] target 1:26 · slot 7:45–9:15

Thanks, Kaiming. Case b is the real test for routing: h2 floods h6 over s2–s6, the link our flow uses. The dashed line is the block.

[click 1: box on 112–116 s] Before detection, s2–s6 carries about 39 megabits of flood plus our 10, on a 40-megabit link. The queue overflows: in four seconds we lose 24, 47, 48 and 27 percent.

[click 2: dashed line at 115.6 s] Then, in one control interval, three things happen together. h2 is blocked. s2–s6 is marked, and its cost jumps to 3.7 times. And our flow moves to s1, s2, s3, s6, because staying would cost 58.8 and moving costs 30. Loss drops to zero.

[click 3: plateau and dotted return line] The flood load is gone, yet the cost holds at 3.14: fourteen plus beta, thirty, over fourteen, as predicted on slide 5, because h2 keeps hitting its drop rule. The flood ends, the flag is held ten more seconds, and about fourteen seconds after the flood the flow comes back. Same shape in all three runs: worst second 34 to 57 percent, zero after the block.

> Return time, three conventions (R1): 39.8 s after flood start; 14.8 s after start + 25 s (the report's 13.6–14.9 s for R1–R3); 14.1 s after the recorded flood end. Say “about 14 seconds after the flood”. Exactly 2 h1→h6 reroutes per run, none in a or c. Throughput and RTT (lowest second 5.41 / 4.55 / 6.74 Mbps; R1 RTT 0.5 → 20.5 ms) are on backup slide 17.

Task 5: Cost, Loss and Throughput over the Whole Run · Slide 11

[Speaker B · Danze] target 0:40 · slot 9:15–10:00

This result confirms slide 5 on every attacked link: a spike at detection, a plateau at C0 plus 30 over C0, then baseline after the hold. Plateaus: 4.0 on s2–s3, 3.14 on s2–s6 in blue, 4.33 and 5.29 on s3–s5 and s3–s6; the same beta lifts a short link more.

Loss and throughput dip only in case b before detection, 48.4 percent and 5.41 megabits; c shows a 2.5 percent blip from h6's access link.

So why is s5–s6 flat, and why did h1 not move in a or c?

> In this figure s2-s6 is drawn in blue because it carries the main path. Links that carry h1 sit a little above 1 (s1-s2 at ×1.54: h1's 10 Mbps adds about 2.7 to C0 = 5). Every marked link is back at its baseline cost 13–15 s after the flood (10 s hold + 1–2 polls). The report's full-size Task 5 plots are on backup slide 17.

Cases a and c: Not Rerouting Is the Correct Answer · Slide 12

[Speaker B · Danze] target 0:29 · slot 10:00–10:30

[click to play, 25 s] A replay of run R1. In a, s2–s3 turns red; our blue path stays, zero loss. In b, our path turns orange and comes back. In c, the flood takes s5, s3, s6; s5–s6 stays grey, and the small loss is h6's access link.

[on the end card] Which part actually earns its place? We switched parts off.

> The replay is paused for 2 s at the case b block (stay 58.8 vs move 30.0). If it will not play: 16 + Enter, then back to 13. The forced s5-s6 run (C0 = 8, plateau 4.75, worst second 35%, also from h6's access link) is backup slide 20.

What Each Part Buys Us: Beta Off, Link Down · Slide 13

[Speaker B · Danze] target 0:28 · slot 10:30–11:00

With beta 0 or 2 the flow stays on s2–s6, yet loss after the block is still zero: blocking ends the loss. Beta guards against what an IP block misses, like spoofed sources.

[click] Cutting s2–s6: rerouted in 7 milliseconds, 11 of 71,515 packets lost; the test also exposed a bug, now fixed.

H4 → So every part earns its place. Where does the remaining loss come from? Kaiming?

> Price of β: the detour costs 3 more (22 vs 19), only while A = 1. Link-down: the link leaves the graph (cost ∞); reroute 7.3 ms after the link_down mark; 10.49 Mbps and 0.06% loss during the outage; the only lossy second is the one with the failure. D13: after a link-down fewer than 11 links were discovered, the controller fell back to “not ready” and ignored ARP, so hosts lost reachability; now ready is latched after the first full discovery.

Where the Loss Lives: Limitations and Takeaways · Slide 14

[Speaker A · Kaiming] target 0:30 · slot 11:00–11:30

Thanks, Danze. Remember the forty-eight percent? Every lost packet fell in the red part of this bar: the 1.5 to 3.8 seconds before detection. So the limitation is detection delay, our weakest link; one-second polling or sFlow would shrink it, at the cost of more control traffic.

[click] **H5 →** And where a source-IP block fails, against spoofed sources, the reroute is the remaining guard. Danze will close.

[Speaker B · Danze] target 0:28 · slot 11:30–12:00

Thanks, Kaiming. Two more limits: the victim's own access link cannot be routed around, and our thresholds are tuned, not derived.

[click] To sum up: in all nine floods, the attacker was blocked within 1.5 to 3.8 seconds. Blocking ends the loss, beta moves traffic off an attacked link, and hysteresis brings it back once. Thank you.

[click: Questions?] Questions on detection, blocking and the setup go to Kaiming; cost, rerouting and the plots to me.

> The Q&A split is spoken only, not on screen. Other limits for Q&A: a partial sample can be learned into the baseline (backup 19); userspace OVS and no link delays in R1–R3 (backup 21).

Case b Demo: Four Stills · Slide 15 (hidden still page)

[Speaker A · Kaiming] target 0:55 · slot replaces slide 8

Use only if the video will not play. Same lines as slide 8, one still each: normal on s1, s2, s6; the flood and the block in one step; the paused frame with the three callouts; tcpdump sees zero attack packets, and the flow is back on the short path. Then the button returns to slide 9.

Replay of Run R1: Four Stills · Slide 16 (hidden still page)

[Speaker B · Danze] target 0:29 · slot replaces slide 12

Use only if the replay will not play. a: s2–s3 marked, our blue path stays, zero loss. b: before detection, then the orange detour while s2–s6 holds at 3.14. c: s3–s5 and s3–s6 marked, s5–s6 grey, our path stays. Then back to slide 13.

Backup: Task 5 Report Plots (R1), Throughput and RTT · Slide 17 (backup, Q&A only)

[Speaker B · Danze] target Q&A · slot backup

For questions on RTT and throughput: before detection s2–s6 carries about 39 Mbps of flood plus our 10 on a 40 Mbps link, so the queue fills. The extra RTT is all queueing; the container has no propagation delay. Mean loss over the 25 s flood, 5.67 / 4.20 / 1.87%, hides that the loss sits in a few seconds.

Backup: Per-Case Before / During / After (R1, with R2 / R3) · Slide 18 (backup, Q&A only)

[Speaker B · Danze] target Q&A · slot backup

For Task 4d/4f and 5.2 questions: path, loss, throughput, cost and recovery for each case. Only case b moves h1→h6, exactly twice per run (away and back).

Backup: Why R1 Case b Took 3.8 s · Slide 19 (backup, Q&A only)

[Speaker A · Kaiming] target Q&A · slot backup

Samples come every 2.4 s (2 s poll plus a fixed 0.4 s wait). The flood began late in an interval, so the first sample saw only part of it: 490.7 pkt/s, under the 500 floor, and it was learned, raising b to 148 and T to 1,046. The next sample, 4,994 pkt/s, was still caught.

Backup: Case c with s5–s6 Forced (C0 = 8) · Slide 20 (backup, Q&A only)

[Speaker B · Danze] target Q&A · slot backup

If asked whether we avoided the assignment's s5–s6 case: we forced it with a cost override. The controller marked s5–s6, h1 did not need to move, and the remaining loss came from h6's own access link, which no reroute can avoid.

Backup: Environment, VM Re-run, Live Demo Commands · Slide 21 (backup, Q&A only)

[Speaker A · Kaiming] target Q&A · slot backup

Routing uses only the C0 table, port statistics and A, so the missing link delays in the container do not change any decision; only absolute RTTs are small. On the VM with netem delays the same behaviour appears, with RTT 99.8 ms normally (98 ms expected) and 109.0 ms on the detour. With link delays the warm-up is 12 samples.

Backup: Parameters, Design Decisions D1–D13, Code Map · Slide 22 (backup, Q&A only)

[Kaiming + Danze] target Q&A · slot backup

Use when asked to point at the code. [Speaker A · Kaiming] owns D2–D6, D8, D11, D12; [Speaker B · Danze] owns D1, D7, D9, D10, D13. Line numbers refer to controller/integrated_controller.py and controller/sdn_logic.py as submitted.

Contributions and Tools · Slide 23 (backup, Q&A only)

[Kaiming + Danze] target Q&A · slot backup

Each person answers in one sentence, matching the two lines on this slide; [Speaker A · Kaiming] first.